FIREWORKS: DEPLOY · MICROSOFT × NVIDIA BUILDATHON

Can cheap open models do frontier-grade business automation?

Holo4-style agents on Zapier's AutomationBench: fine-tuned open models, a Holo-style harness, and a learned per-task router, all served on Fireworks.

Benchmark
600
tasks across 47 simulated apps

Bar to beat
Kimi K3
~47% pass on our held-out set

Spend so far
~$22
for two LoRA fine-tunes

The benchmark: AutomationBench

Zapier, v1.0.6

- A business request, e.g. *"pull the invoices from email, log them in the AP sheet, post the total to #finance"*.
- The agent has **3 tools**: `api_search` over 758 endpoints, `api_fetch`, and `base64_encode`. It gets at most 50 steps.
- Graded **only on the final app state**, using a median of 11 assertions per task. About 39% of them are "don't do X" traps.
- A task counts as passed only if **every** assertion passes. Partial credit is reported but isn't the headline number.

Hard because the context is spread out: corrections hide in Slack, policies in Drive, and stale rows in Sheets.

Published leaderboard (public 600)

Model	Pass	$/task
Claude Opus 5	50.3%	$3.05
Kimi K3	46.7%	$0.43
Holo4 27B (H Company)	45.4%	**$0.05**
Holo4 35B-A3B	34.5%	$0.02

Holo4 shows that a small trained model can come close to the frontier at about 1/60 of the cost. That's our inspiration.

Our approach: three cheap levers, one router

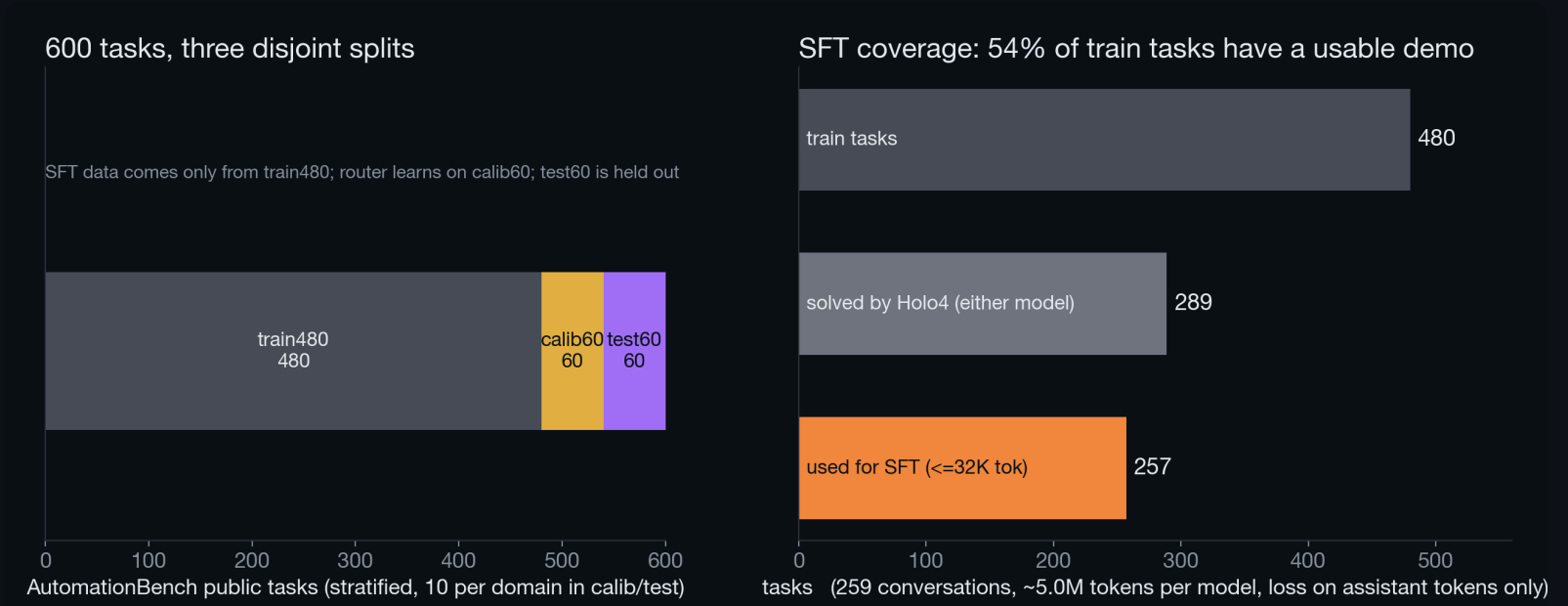


1 · Distill Holo4. LoRA SFT of Qwen3.6-35B-A3B and Nemotron 3.5 Super on H Company's successful trajectories.

2 · Harness. Holo's operating procedure plus a "verify before you finish" gate on DeepSeek V4.1 Flash.

3 · Route. A kNN over task embeddings picks the system with the best score on similar tasks, breaking ties by lower cost.

Honest evaluation by design



No leakage

SFT uses only train480 trajectories (checked by an assertion). The router is fit on calib60 and scored on test60.

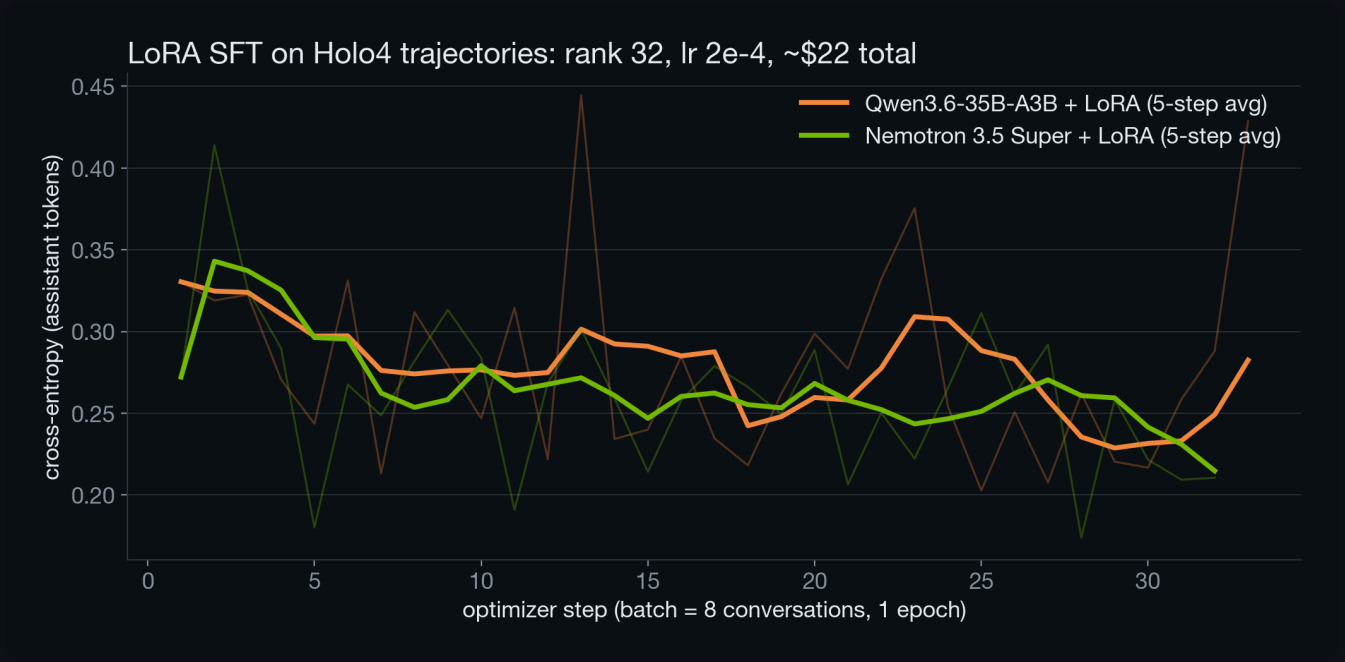
Official metric

All-assertions pass rate, plus avg partial credit and **real $/task** from token usage with cache pricing.

Paired stats

Exact McNemar on the same tasks, plus 95% bootstrap CIs. At n=60 the CI is about ±12 pts, so we don't oversell small gaps.

Lever 1: distilling Holo4 with LoRA SFT



- Holo4 trajectories were converted to AutomationBench's exact tool schema and system prompt, keeping reasoning and tool calls.
- **259 conversations, ~5.0M tokens** per model, ≤32K tokens each. Loss is computed on assistant tokens only.
- LoRA rank 32, lr 2e-4 with cosine decay, 1 epoch, on **Fireworks serverless training**: no GPUs to provision, billed per token.
- Qwen **$10.68**, Nemotron **$11.59**.
- Served through a small OpenAI-compatible proxy over the serverless sampler, so the stock benchmark runner can call it.

Lever 2: a Holo-style harness

Operating procedure (appended to the system prompt)

1. **Orient**: find policies, corrections, and updates in every app first.
2. **Plan**: write an explicit checklist of the changes to make.
3. **Search before fetch**: never guess an endpoint.
4. **Chain**: copy IDs, amounts, and formats verbatim.
5. **Recover**: never repeat a failing call.
6. **Mutate precisely**: stray changes cost as much as missing ones.
7. **Verify**, then finish.

Verify-before-finish gate

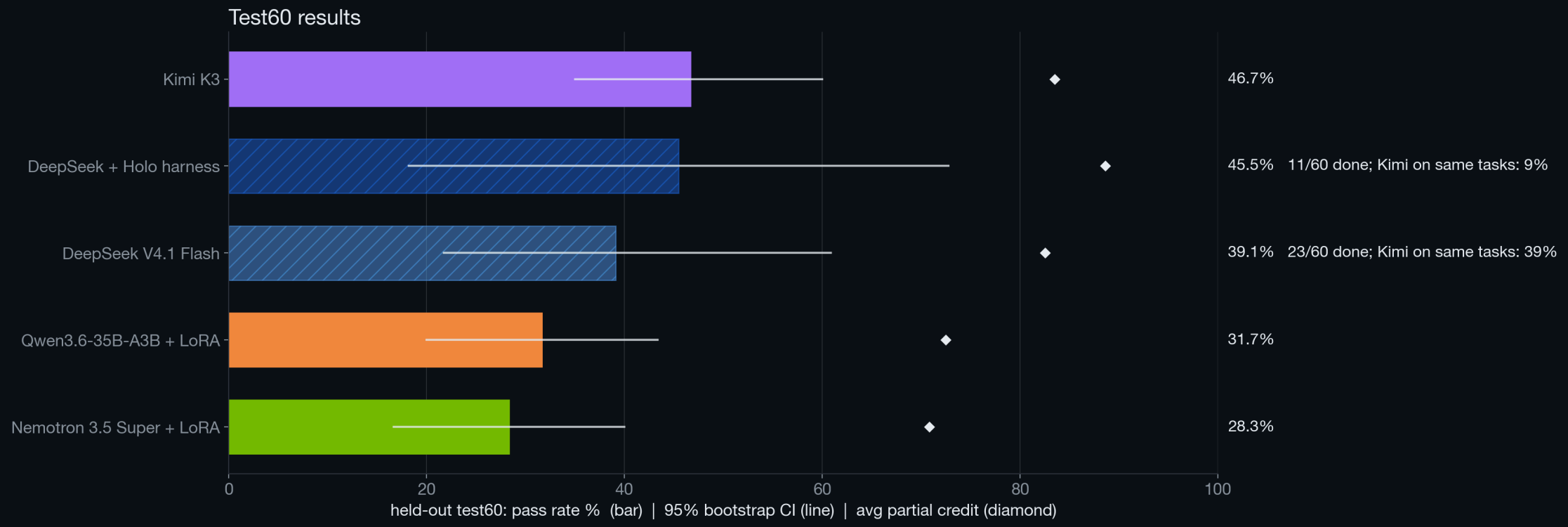
The first time the model tries to stop, the harness injects a checklist alert. The model must re-read the request and policies and check the final state with read calls.

Why it matters

In the Kimi smoke run, Kimi missed a Slack correction ($12,340.50 → $11,340.50) and shifted columns in a sheet. That's 3/5 assertions lost on one task, from *process* errors rather than a lack of knowledge.

Results on held-out test60

live · DeepSeek V4.1 Flash 23/60 · DeepSeek + harness 11/60



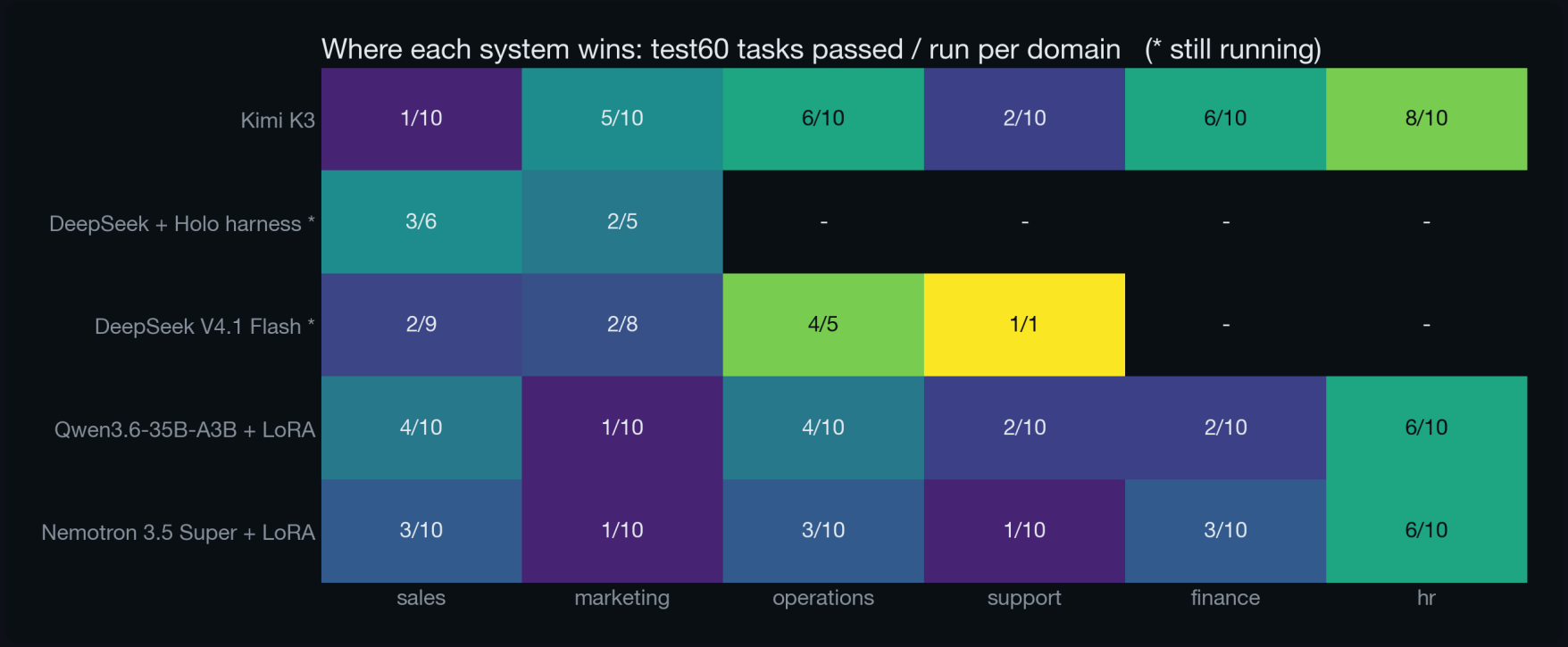
On the same 60 tasks, Kimi K3 beats Qwen LoRA (6 vs 15 discordant tasks, McNemar p=0.078) and Nemotron LoRA (5 vs 16 discordant tasks, McNemar p=0.027). DeepSeek runs are partial; compare them with Kimi on the same tasks (shown at right of each bar), not with Kimi's full-60 bar.

Pass rate vs cost



- **DeepSeek V4.1 Flash** costs about **$0.022–0.036**/task, **4–6× cheaper than Kimi K3** on our runs. Prompt caching covers 99%+ of its input tokens.
- **LoRA models** cost $0.10–0.16/task *only because* the serverless sampler we used for eval has no cache discount. A dedicated deployment with prompt caching would cut that sharply.
- Published points use the public 600 tasks and other harnesses, so treat them as context, not a head-to-head.

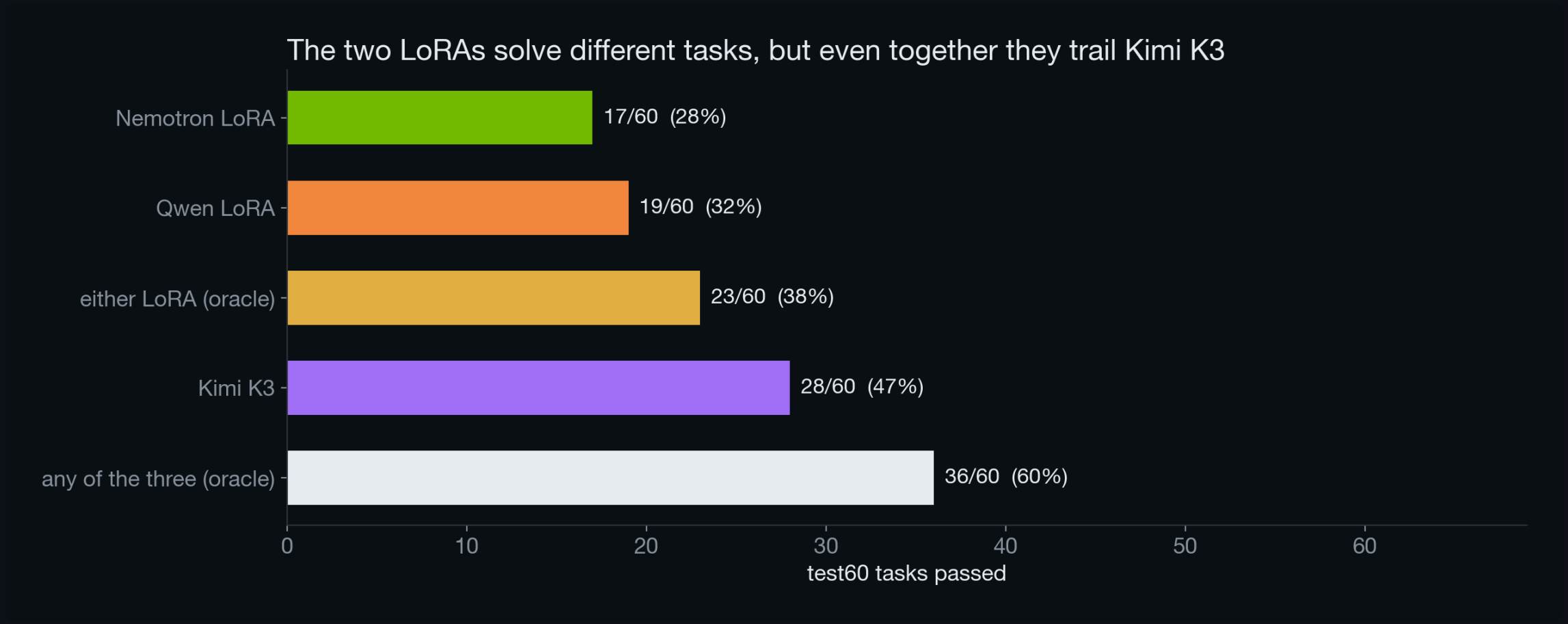
Where each system wins



Kimi K3 leads in HR, finance, and ops, but gets **1/10 in sales**. Both LoRAs beat it there (4/10 and 3/10).

Domain strengths differ between systems. A router can exploit that, but only if it can predict them from the task text.

Complementarity: the case for a router



Each LoRA solves tasks the other misses, but the best possible pick between them (23/60) still trails Kimi K3 alone (28/60). Adding Kimi raises the ceiling to 36/60, so routing pays off only if a stronger, cheap member joins the pool. DeepSeek + harness is the candidate.

So, do the results look good?

Not yet: distillation alone falls short

- Qwen LoRA **31.7%** and Nemotron LoRA **28.3%** vs Kimi K3 **46.7%** on test60. The Nemotron gap is significant (p=0.027); Qwen's is borderline (p=0.078).
- They aren't cheaper yet either: $0.10–0.16/task on the uncached serverless sampler vs Kimi's $0.14.
- We haven't measured base Qwen/Nemotron on test60 yet, so the size of the SFT lift is still unknown.

Promising: the cheap base + harness

- DeepSeek base ties Kimi on the same 23 tasks (2 vs 2 discordant) at **~6× lower cost**.
- With the harness: 11 tasks done, 45.5% pass vs Kimi 9.1% on the same tasks (4–0). Early signal.
- The final verdict waits on the DeepSeek runs and the router.

Lessons worth sharing

- **Small screens mislead.** DeepSeek scored 66.7% on the 30-task screen but only 39.1% on test60 so far. Hold out tasks and compare systems on the same ones.
- **Grader quirk.** The Slack matcher rejects `12,450.00` when it expects `12,450`, while Gmail and Sheets accept it. Don't pad amounts with ".00" in Slack.
- **Coverage caps SFT.** Holo4 solved only 289/480 train tasks, so the fine-tunes never saw demos for ~40% of task types.
- **Cost depends on caching.** Agent loops resend about 150K tokens per task. Prompt caching makes a 5× difference to $/task.

Next steps

Finish the DeepSeek runs

Complete base and harness on calib60 and test60, then fit the router over {DeepSeek+harness, Qwen LoRA, Nemotron LoRA} and report it with McNemar against Kimi.

Better distillation

Distill DeepSeek+harness's own successes on train480 to cover tasks Holo4 failed. Add the harness procedure to the SFT prompts.

Cheaper serving

Deploy the LoRA on a Fireworks on-demand deployment with prompt caching, then re-price. That brings the LoRA models' $/task into DeepSeek's range.

Reproduce: `python3 slides/make_figures.py` rebuilds every chart and number in this deck from `runs/`. Code: `github.com/abhishm/fireworks-buildathon-holo4`